

Skill 11 os

SESSION 21

TASK 1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 s11  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
int validate_command(char *cmd){  
    if(cmd==NULL || strlen(cmd)==0){  
        printf("Error: Empty command.\n");  
        return 0;  
    }  
    return 1;  
}  
  
int execute_command(char *cmd){  
    printf("Executing: %s\n",cmd);  
    int result=system(cmd);  
    if(result==-1){  
        printf("Error: Command execution failed.\n");  
        return 0;  
    }  
    return 1;  
}  
  
void display_result(int success){  
    if(success)  
        printf("Execution completed successfully.\n");  
    else  
        printf("Execution failed.\n");  
}  
  
int main(){  
    char command[100];  
    int valid,success;  
  
    printf("=====\n");  
    printf("      MODULE INTEGRATION TEST\n");  
    printf("=====\n");  
  
    printf("Enter command: ");  
    fgets(command,sizeof(command),stdin);  
    command[strcspn(command,"\n")]='\0';  
  
    valid=validate_command(command);  
  
    if(valid){  
        success=execute_command(command);
```

```

    valid=validate_command(command);

    if(valid){
        success=execute_command(command);
        display_result(success);
    }else{
        printf("Invalid input. Program stopped safely.\n");
    }

    return 0;
}

```

Output:

```

=====
          MODULE INTEGRATION TEST
=====
Enter command: ls
Executing: ls
a.out      error.log  s10.c     s10T4.c   s72.c    s81.c    s84.c    s93.c
combined.log  log.txt   s10T2.c   s11T1.c   s73.c    s82.c    s91.c    s94.c
combined2.log output.txt s10T3.c   s71.c     s74.c    s83.c    s92.c
Execution completed successfully.

```

Observation: The modules were successfully integrated and tested end-to-end with input validation and error handling.

TASK 2: To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

Code:

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

void log_error(char *message){
    FILE *file=fopen("error.log","a");
    if(file!=NULL){
        fprintf(file,"ERROR: %s\n",message);
        fclose(file);
    }
}

int main(){
    char command[100];
    int result;

    printf("=====\n");
    printf("      ERROR HANDLING TEST\n");
    printf("=====\n");

    while(1){
        printf("\nEnter command (exit to quit): ");
        fgets(command,sizeof(command),stdin);
        command[strcspn(command,"\n")]='\0';

        if(strcmp(command,"exit")==0)
            break;

        if(strlen(command)==0){
            printf("Error: Empty command.\n");
            log_error("Empty command entered.");
            continue;
        }

        if(strchr(command,';')!=NULL){
            printf("Error: Invalid syntax.\n");
            log_error("Invalid syntax detected.");
            continue;
        }

        printf("Executing: %s\n",command);

        result=system(command);
    }
}
```

```

        if(result==-1){
            printf("Runtime error: Command execution failed.\n");
            log_error("Runtime command execution failed.");
        }
        else if(result!=0){
            printf("Error: Command failed with status %d.\n",result);
            log_error("Command returned an error status.");
        }
        else{
            printf("Command executed successfully.\n");
        }
    }

    printf("\nProgram recovered safely and exited.\n");
    return 0;
}

```

Output:

```

=====
ERROR HANDLING TEST
=====

Enter command (exit to quit): abcdefg
Executing: abcdefg
sh: 1: abcdefg: not found
Error: Command failed with status 32512.

Enter command (exit to quit): ls;
Error: Invalid syntax.

Enter command (exit to quit): ls
Executing: ls
a.out      error.log  s10.c     s10T4.c   s71.c     s74.c     s83.c     s92.c
combined.log  log.txt   s10T2.c   s11T1.c   s72.c     s81.c     s84.c     s93.c
combined2.log output.txt s10T3.c   s11T2.c   s73.c     s82.c     s91.c     s94.c
Command executed successfully.

Enter command (exit to quit): exit
Program recovered safely and exited.

```

Observation: The program detects runtime and syntax errors, logs failures, displays suitable error messages, and continues execution without crashing.

SESSION 22

TASK 1: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

Code:

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(5 * sizeof(int));

    if (ptr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;

    printf("Values: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);

    printf("\nMemory allocated successfully.\n");

    free(ptr);
    ptr = NULL;

    printf("Memory released successfully.\n");

    return 0;
}

```

Output:

```

Values: 1 2 3 4 5
Memory allocated successfully.
Memory released successfully.

```

Observation:

Memory was successfully allocated using malloc() and released using free(). Valgrind confirmed that there were **no memory leaks or memory errors**, verifying proper memory cleanup.

TASK 2: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 s1  
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    int result;  
  
    printf("Test Case 1: Addition\n");  
    result = 10 + 20;  
    printf("Expected: 30\nActual: %d\n", result);  
    printf("%s\n\n", result == 30 ? "PASS" : "FAIL");  
  
    printf("Test Case 2: Multiplication\n");  
    result = 5 * 4;  
    printf("Expected: 20\nActual: %d\n", result);  
    printf("%s\n\n", result == 20 ? "PASS" : "FAIL");  
  
    printf("Test Case 3: Subtraction\n");  
    result = 20 - 5;  
    printf("Expected: 15\nActual: %d\n", result);  
    printf("%s\n", result == 15 ? "PASS" : "FAIL");  
  
    return 0;  
}
```

Output:

```
Test Case 1: Addition  
Expected: 30  
Actual: 30  
PASS  
  
Test Case 2: Multiplication  
Expected: 20  
Actual: 20  
PASS  
  
Test Case 3: Subtraction  
Expected: 15  
Actual: 15  
PASS
```

Observation: Test cases were created and executed automatically. Expected and actual outputs were compared, and each test was reported as **PASS**, demonstrating basic test automation and result verification.